

MANUAL DO PASID-VALIDATOR



IDEIA GERAL

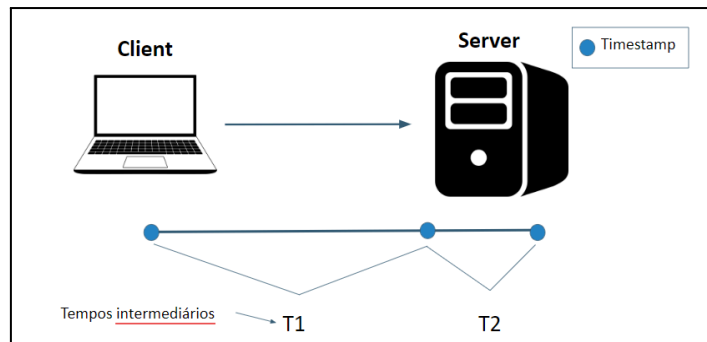
Validar um modelo de **Stochastic Petri Net (SPN)** com experimentos significa verificar se o modelo teórico (SPN) representa adequadamente o sistema real que ele tenta modelar, e se as suas previsões ou simulações são consistentes com os dados observados ou resultados experimentais. A validação de um modelo SPN envolve a comparação entre o comportamento do modelo (como tempos de espera, taxas de processamento, ou qualquer outra métrica de interesse) e o comportamento de um sistema real ou os resultados de experimentos controlados. Aqui estão os passos principais para validar um modelo SPN com experimentos:

1. **EXECUÇÃO EXPERIMENTAL DE COLETA:** Coleta de dados experimentais para alimentar o modelo (feeding). Esses dados servirão como referência para validar o comportamento do modelo. Aqui você monta seu experimento e coleta os “tempos intermediários” que serão inseridos no modelo.
2. **EXECUÇÃO DO MODELO:** Simular o modelo SPN para gerar dados de saída baseados nos tempos intermediários.
3. **EXECUÇÃO EXPERIMENTAL PARA VALIDAÇÃO:** Rodar o experimento variando algum parâmetro como a quantidade de recursos paralelos.
4. **COMPARAÇÃO DOS RESULTADOS DAS ETAPAS 2 E 3:** Comparar os resultados das simulações do modelo com os dados obtidos dos experimentos reais usando o teste t.

NOMENCLATURA DOS TEMPOS

Por vários motivos usamos a métrica MRT para validação.

A imagem abaixo mostra a ideia dos tempos coletados.



Timestamps (marcos temporais): Representados pelos pontos azuis ao longo do caminho entre o cliente e o servidor. Esses pontos indicam os momentos em que os tempos são registrados, como:

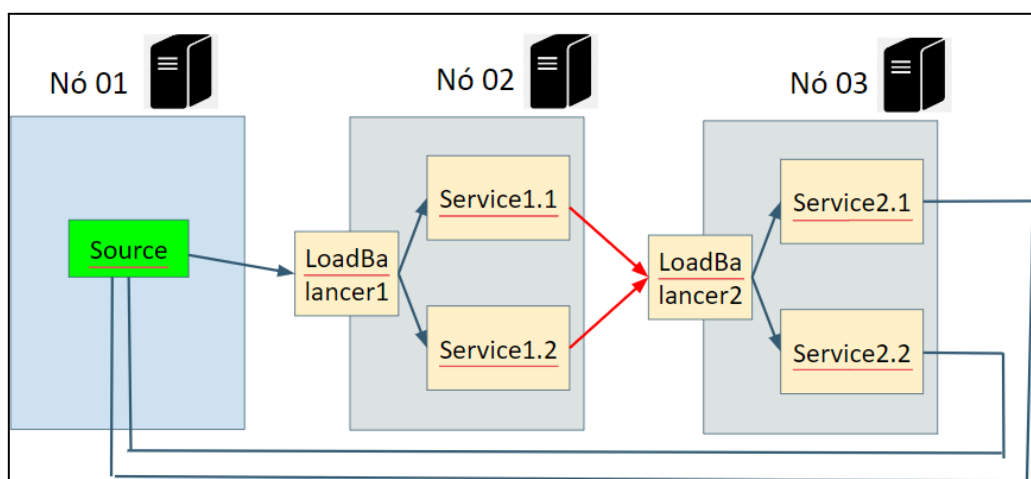
- O tempo que a mensagem leva para sair do cliente até o servidor.
- O tempo que a mensagem leva para voltar do servidor até o cliente.

Tempos intermediários: Estes tempos são tempos de cada etapa. A linha destacada mostra dois momentos de interesse no caminho: o T1 e o T2. Esses pontos são tempos intermediários que podem representar, por exemplo:

- T1: O momento em que o servidor recebe a solicitação do cliente.
- T2: O momento em que o servidor processa e envia a resposta de volta ao cliente.

EXEMPLO COM TRÊS NÓS

A imagem abaixo representa um sistema distribuído com três nós, cada um contendo diferentes componentes que trabalham em conjunto.



1. Nó 01 - Source:

- O primeiro nó (Nó 01) contém um componente chamado **Source**, representado em verde, responsável por gerar ou fornecer **dados** para o sistema. Ele é o ponto de origem de onde as solicitações ou mensagens são enviadas para processamento e também o componente que irá compilar os resultados e fazer o cálculo final da validação.

2. Nó 02 - LoadBalancer1 e Serviços:

- O **Nó 02** possui um **LoadBalancer1**, responsável por distribuir as solicitações recebidas do **Source** entre dois serviços: **Service1.1** e **Service1.2**.
- O **LoadBalancer1** atua como um balanceador de carga, direcionando as tarefas de forma balanceada ou de acordo com algum critério específico, para os serviços de processamento.
- O **Nó 02** é responsável por uma parte do processamento, onde os serviços 1.1 e 1.2 recebem suas respectivas cargas de trabalho.

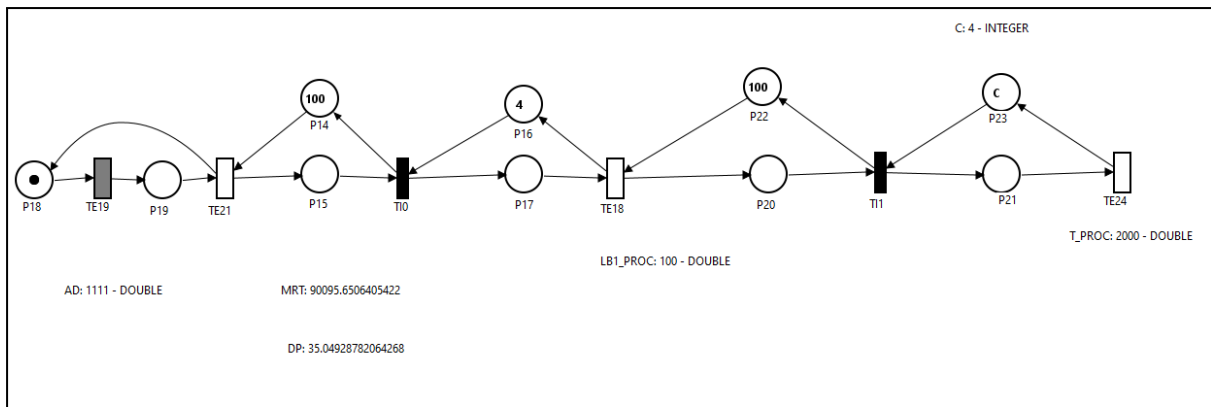
3. Nó 03 - LoadBalancer2 e Serviços:

- O **Nó 03** possui um **LoadBalancer2**, que distribui as solicitações recebidas entre dois serviços: **Service2.1** e **Service2.2**.
- As setas mostram que **LoadBalancer2** recebe solicitações vindas do **Nó 02** (os serviços 1.1 e 1.2), indicando que, após o processamento inicial no **Nó 02**, as tarefas são encaminhadas para o **Nó 03** para continuar o processamento.

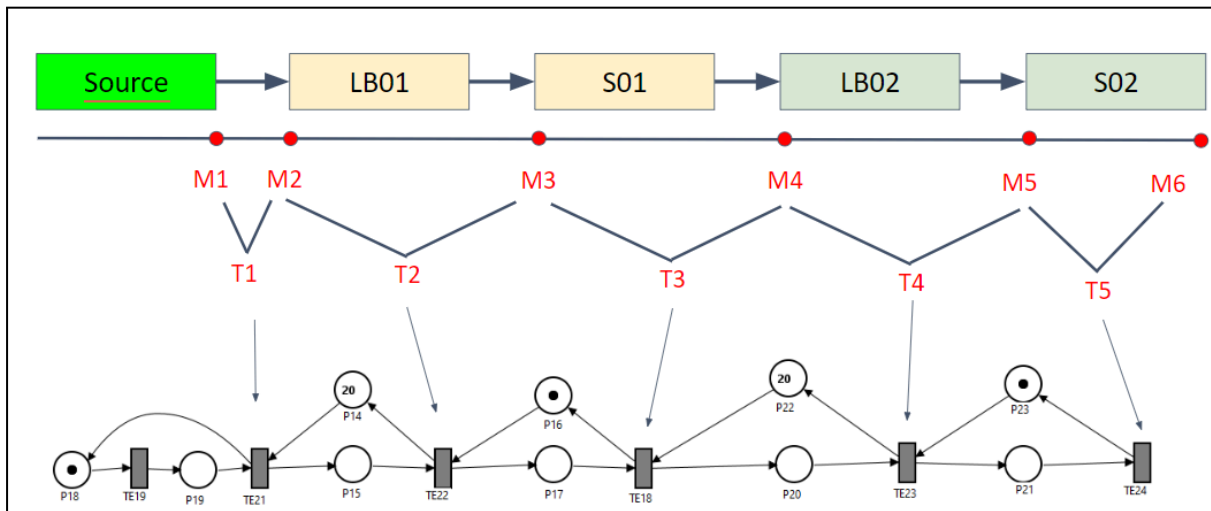
4. Fluxo de Comunicação:

- As **setas** no diagrama indicam o fluxo de comunicação:
 - O **Source** no **Nó 01** envia solicitações para o **LoadBalancer1** no **Nó 02**.
 - O **LoadBalancer1** distribui essas solicitações entre **Service1.1** e **Service1.2**.
 - Após o processamento inicial, as tarefas são enviadas para o **LoadBalancer2** no **Nó 03**.
 - O **LoadBalancer2** distribui as tarefas entre **Service2.1** e **Service2.2** para um processamento adicional.
 - O sistema segue um fluxo encadeado onde os dados fluem de um nó para outro, com balanceadores de carga gerenciando a distribuição de trabalho em cada etapa.

A imagem abaixo mostra o modelo correspondente a ser validado, com dois loadbalancers. A variação ocorrerá no parâmetro C do segundo loadbalancer. Alguns tempos de distribuição para os services são desprezados, restando os tempos de transmissão entre source e LB e entre LB/processamento do LB. O AD é determinístico mas pode ser exponencial.



A imagem abaixo mostra o mapeamento de tempos intermediários ao modelo.



PASID VALIDATOR

O PASID-VALIDATOR é uma ferramenta para auxiliar este processo de validação de modelos analíticos.

BAIXANDO E CONFIGURANDO O PROJETO

Link para acessar o código:

<https://gitlab.com/pasid3/pasid-validator.git>

O validador usa o mercuryauto, logo é bom usar a versão do java que não dá erro, a zulu
Va em File - Project Structure...

Instalar a seguinte versão do Java 11:

WINDOWS:

zulu11.68.17-ca-fx-jdk11.0.21-win_x64

https://cdn.azul.com/zulu/bin/zulu11.68.17-ca-fx-jdk11.0.21-win_x64.msi

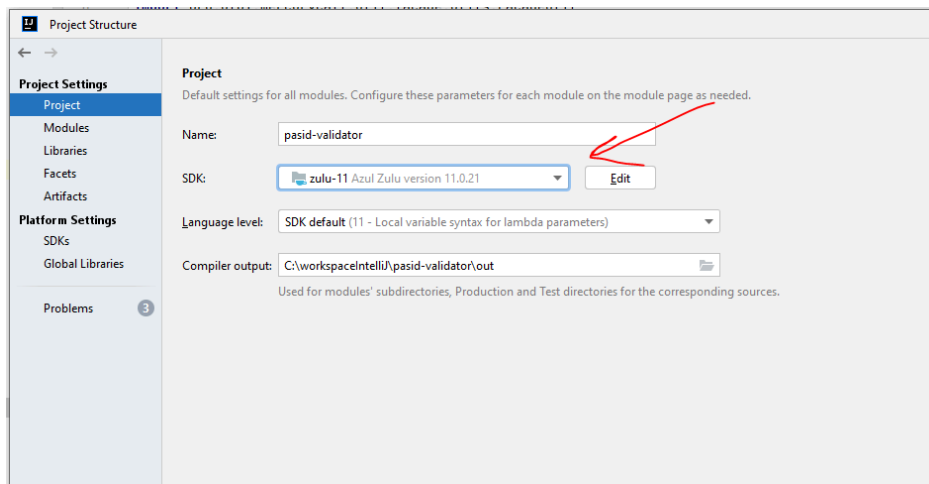
LINUX

zulu11.68.17-ca-fx-jdk11.0.21-linux_amd64

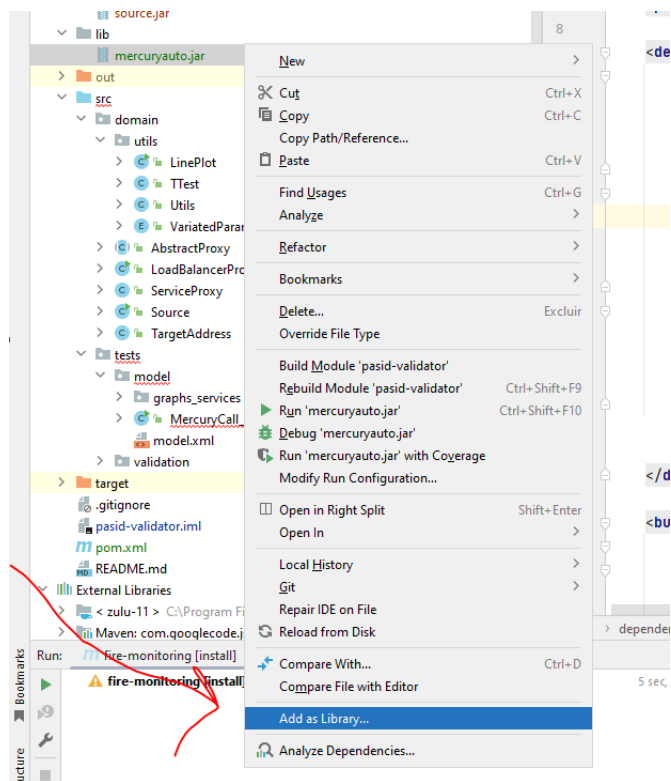
https://cdn.azul.com/zulu/bin/zulu11.68.17-ca-fx-jdk11.0.21-linux_amd64.deb

Depois é só apontar no seu projeto

File > Project Structure > Project Settings > Project

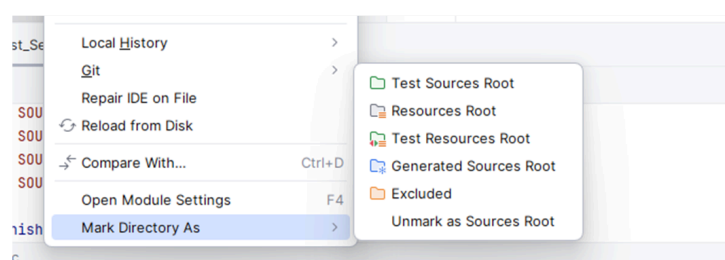
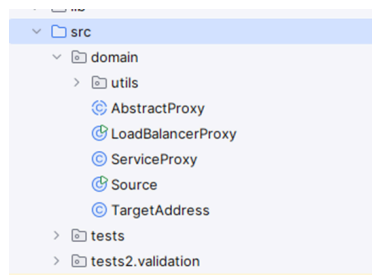


Adicione a lib do mercuryauto no braço, coloque o jar na pasta lib e botão direito do mouse e add lib como mostra a imagem abaixo



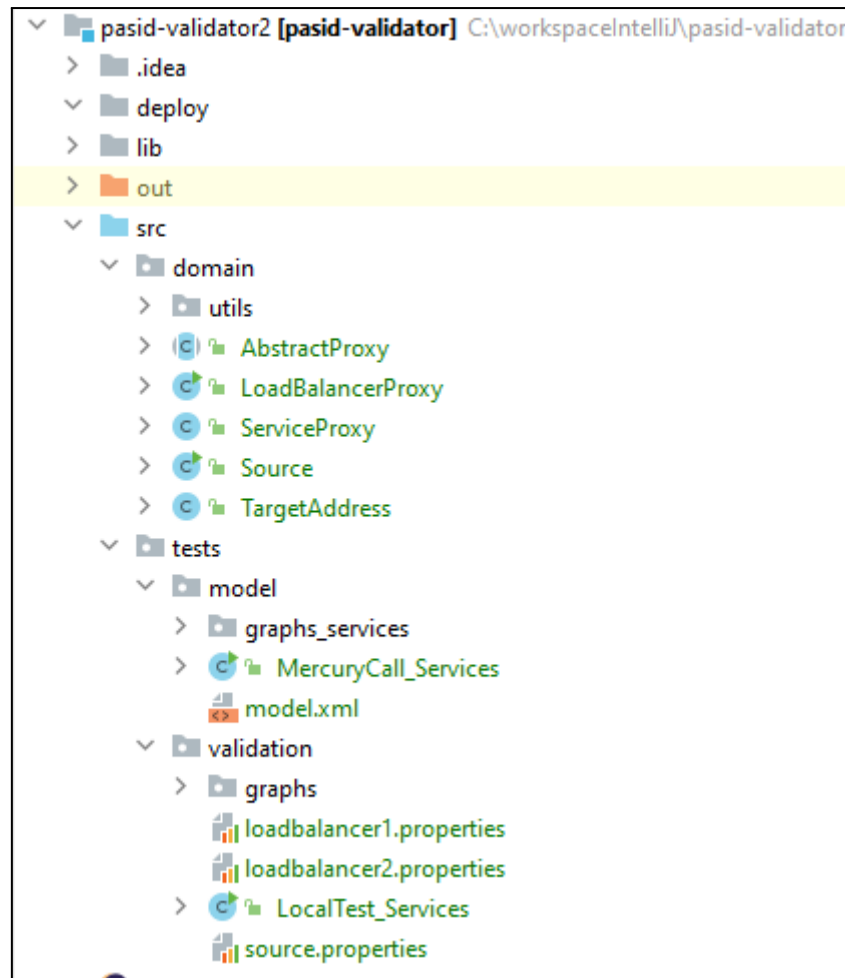
Jar do mercuryaudio:

https://drive.google.com/file/d/1cAj5S3gQAIVwcHd4YdM0Qk_7LwO_OJmV/view?usp=sharing



ENTENDENDO O PROJETO

A imagem abaixo mostra o layout do projeto.



Vamos focar na pasta tests. Aqui você encontra as pastas model e validation. Primeiro observe model, onde temos o modelo xml em si e a classe MercuryCall_Services que executa o modelo e gera gráficos na pasta graphs_services.

A pasta validation é onde executa o experimento em si. A classe LocalTest_Services lê os três arquivos properties e executa a validação.

Aqui está uma explicação do que esses tipos de arquivos representam e como eles são usados no contexto de sistemas distribuídos e balanceadores de carga:

loadbalancer2.properties

Este arquivo é um arquivo de **propriedades** que contém as configurações específicas para o segundo balanceador de carga no seu sistema (o **LB02**). Essas configurações podem incluir:

- **Limites de carga:** O número máximo de solicitações ou conexões que o balanceador pode distribuir.

- **Política de balanceamento:** Definições sobre como o balanceador distribui as requisições entre os serviços. Pode ser por round-robin, menor carga, ou com base em métricas específicas.
- **Tempos de resposta:** Configurações relacionadas ao tempo de espera ou timeout, que podem definir quanto tempo o balanceador deve aguardar uma resposta antes de redistribuir uma solicitação.
- **Parâmetros de rede:** Detalhes de configuração da rede, como endereços IP ou portas que o balanceador utiliza para se conectar aos serviços.

Esses parâmetros controlam o comportamento do balanceador, garantindo que ele distribua as cargas de forma eficiente e dentro dos limites aceitáveis.

source.properties

Este arquivo de propriedades está relacionado ao componente **Source**, que é responsável por gerar ou fornecer solicitações ao sistema. As configurações nesse arquivo podem incluir:

- **Taxa de geração de solicitações:** A frequência com que o Source gera novas solicitações ou mensagens a serem distribuídas pelo sistema.
- **Destinos das solicitações:** Os endereços ou serviços para os quais as solicitações serão enviadas.
- **Parâmetros de conexão:** Configurações sobre como o Source se conecta aos serviços, como IPs, portas, e timeout.
- **Parâmetros de simulação:** Se o Source estiver simulando um comportamento de entrada de dados, ele pode conter parâmetros para ajustar a carga gerada ou o tipo de tráfego simulado.

Esses parâmetros permitem que o Source ajuste dinamicamente a carga de trabalho enviada para os balanceadores e serviços no sistema, o que pode ser útil para testar e validar o comportamento do sistema sob diferentes condições de carga.

Agora vamos voltar às etapas explicadas no início deste documento

1. **EXECUÇÃO EXPERIMENTAL DE COLETA:** Alterar a variável `modelFeedingStage` para true e executar LocalTestServices

O resultado sai assim:

The times to feed the models transitions are the following:

T1 = 226.33333333333334

T2 = 72.0

T3 = 177.0

T4 = 13.333333333333334

T5 = 2030.333333333333

Estes são os tempos que você deve alimentar as transições do modelo.

2. **EXECUÇÃO DO MODELO:** Simular o modelo SPN para gerar dados de saída baseados nos tempos intermediários.

Com estes tempos intermediários alimenta o modelo (não precisa immediate transitions) e o executa com a classe MercuryCall_Service.

A saída é justamente os dados que irá colocar no arquivo source.properties para a próxima etapa:

```
variatingServices.qtdServices=1,2,3,4
```

```
mrtsFromModel=405597.23,203892.96,136603.74,102902.92
```

```
sdvsFromModel=1245.97,613.95,400.11,313.47
```

3. **EXECUÇÃO EXPERIMENTAL PARA VALIDAÇÃO:** Rodar o experimento variando algum parâmetro como a quantidade de recursos paralelos.

Aqui executar novamente LocalTest_Services e no log aparecerá a tabela com os resultados do teste t

Falta colocar o log aqui

4. **COMPARAÇÃO DOS RESULTADOS DAS ETAPAS 2 E 3:** Comparar os resultados das simulações do modelo com os dados obtidos dos experimentos reais usando o teste t.

Obviamente as classes da pasta teste são para teste local.